Code Smells

Carsten Gips (HSBI)

TL;DR
Code entsteht nicht zum Selbstzweck, er muss von anderen Menschen leicht verstanden und gewartet werden können: Entwickler verbringen einen wesentlichen Teil ihrer Zeit mit dem **Lesen** von (fremdem) Code. Dabei helfen "Coding Conventions", die eine gewisse einheitliche äußerliche Erscheinung des Codes vorgeben (Namen, Einrückungen, ...). Die Beachtung von grundlegenden Programmierprinzipien hilft ebenso, die Lesbarkeit und Verständlichkeit zu verbessern.
Code, der diese Konventionen und Regeln verletzt, zeigt sogenannte "**Code Smells**" oder "Bad Smells". Das sind Probleme im Code, die noch nicht direkt zu einem Fehler führen, die aber im Laufe der Zeit die Chance für echte Probleme deutlich erhöht.

Videos

- VL Code Smells

Unterlagen

- PDF-Version

---

1 Code Smells: Ist das Code oder kann das weg?

```
class checker {
    static public void CheckANDDO(DATA1 inp, int c, FH.Studi
    CustD, int x, int y, int in, int out,int c1, int c2, int c3 = 4)
{
    public int i; // neues i
for(i=0;i<10;i++) // fuer alle i
{
        inp.kurs[0] = 10; inp.kurs[i] = CustD.cred[i]/c;
}
      SetDataToPlan(  CustD  );
    public double myI = in*2.5; // myI=in*2.5
    if (c1)
        out = myI; //OK
    else if(  c3 == 4  )
    {
        myI = c2 * myI;
    if (c3 != 4 || true ) { // unbedingt beachten!
        //System.out.println("x:"+(x++));
        System.out.println("x:"+(x++)); // x++
        System.out.println("out: "+out);
    } }}   }
```

Der Code im obigen Beispiel lässt sich möglicherweise kompilieren. Und möglicherweise tut er sogar das, was er tun soll.

Dennoch: **Der Code "stinkt"** (zeigt **Code Smells**):

- Nichtbeachtung üblicher Konventionen (Coding Rules)
- Schlechte Kommentare

- Auskommentierter Code
- Fehlende Datenkapselung
- Zweifelhafte Namen
- Duplizierter Code
- “Langer” Code: Lange Methoden, Klassen, Parameterlisten, tief verschachtelte `if`/`then`-Bedingungen, …
- Feature Neid
- `switch`/`case` oder `if`/`else` statt Polymorphie
- Globale Variablen, lokale Variablen als Attribut
- Magic Numbers

Diese Liste enthält die häufigsten “Smells” und ließe sich noch beliebig fortsetzen. Schauen Sie mal in die unten angegebene Literatur :-)

Stinkender Code führt zu möglichen (späteren) Problemen.

2 Was ist guter (“sauberer”) Code (“Clean Code”)?

Im Grunde bezeichnet “sauberer Code” (“Clean Code”) die Abwesenheit von Smells. D.h. man könnte Code als “sauberen” Code bezeichnen, wenn die folgenden Eigenschaften erfüllt sind (keine vollständige Aufzählung!):

- Gut (“angenehm”) lesbar
- Schnell verständlich: Geeignete Abstraktionen
- Konzentriert sich auf **eine** Aufgabe
- So einfach und direkt wie möglich
- Ist gut getestet

In (Martin 2009) lässt der Autor Robert Martin verschiedene Ikonen der SW-Entwicklung zu diesem Thema zu Wort kommen - eine sehr lesenswerte Lektüre!

=> Jemand kümmert sich um den Code; solides Handwerk

3 Warum ist guter (“sauberer”) Code so wichtig?

> Any fool can write code that a computer can understand. Good programmers write code that humans can understand.
>
> Quelle: (Fowler 2011, p. 15)

Auch wenn das zunächst seltsam klingt, aber Code muss auch von Menschen gelesen und verstanden werden können. Klar, der Code muss inhaltlich korrekt sein und die jeweilige Aufgabe erfüllen, er muss kompilieren etc. … aber er muss auch von anderen Personen weiter entwickelt werden und dazu gelesen und verstanden werden. Guter Code ist nicht einfach nur inhaltlich korrekt, sondern kann auch einfach verstanden werden.

Code, der nicht einfach lesbar ist oder nur schwer verständlich ist, wird oft in der Praxis später nicht gut gepflegt: Andere Entwickler haben (die berechtigte) Angst, etwas kaputt zu machen und arbeiten "um den Code herum". Nur leider wird das Konstrukt dann nur noch schwerer verständlich …

3.1 Code Smells

Verstöße gegen die Prinzipien von *Clean Code* nennt man auch *Code Smells*: Der Code "stinkt" gewissermaßen. Dies bedeutet nicht unbedingt, dass der Code nicht funktioniert (d.h. er kann dennoch compilieren und die Anforderungen erfüllen). Er ist nur nicht sauber formuliert, schwer verständlich, enthält Doppelungen etc., was im Laufe der Zeit die Chance für tatsächliche Probleme deutlich erhöht.

Und weil es so wichtig ist, hier gleich noch einmal:

Stinkender Code führt zu möglichen (späteren) Problemen.

3.2 "Broken Windows" Phänomen

Wenn ein Gebäude leer steht, wird es eine gewisse Zeit lang nur relativ langsam verfallen: Die Fenster werden nicht mehr geputzt, es sammelt sich Graffiti, Gras wächst in der Dachrinne, Putz blättert ab …

Irgendwann wird dann eine Scheibe eingeworfen. Wenn dieser Punkt überschritten ist, beschleunigt sich der Verfall rasant: Über Nacht werden alle erreichbaren Scheiben eingeworfen, Türen werden zerstört, es werden sogar Brände gelegt …

Das passiert auch bei Software! Wenn man als Entwickler das Gefühl bekommt, die Software ist nicht gepflegt, wird man selbst auch nur relativ schlechte Arbeit abliefern. Sei es, weil man nicht versteht, was der Code macht und sich nicht an die Überarbeitung der richtigen Stellen traut und stattdessen die Änderungen als weiteren "Erker" einfach dran pappt. Seit es, weil man keine Lust hat, Zeit in ordentliche Arbeit zu investieren, weil der Code ja eh schon schlecht ist … Das wird mit der Zeit nicht besser …

"Broken Windows" Phänomen

3.3 Maßeinheit für Code-Qualität ;-)

Es gibt eine "praxisnahe" (und nicht ganz ernst gemeinte) Maßeinheit für Code-Qualität: Die "WTF/m" (*What the Fuck per minute*): Thom Holwerda: www.osnews.com/story/19266/WTFs_.

Wenn beim Code-Review durch Kollegen viele "WTF" kommen, ist der Code offenbar nicht in Ordnung …

4 Code Smells: Nichtbeachtung von Coding Conventions

- Richtlinien für einheitliches Aussehen => Andere Programmierer sollen Code schnell lesen können

  - Namen, Schreibweisen
  - Kommentare (Ort, Form, Inhalt)
  - Einrückungen und Spaces vs. Tabs
  - Zeilenlängen, Leerzeilen
  - Klammern

- Beispiele: Sun Code Conventions, Google Java Style

- *Hinweis*: Betrifft vor allem die (äußere) Form!

5 Code Smells: Schlechte Kommentare I

- Ratlose Kommentare

```
/* k.A. was das bedeutet, aber wenn man es raus nimmt, geht's nicht mehr */
/* TODO: was passiert hier, und warum? */
```

Der Programmierer hat selbst nicht verstanden (und macht sich auch nicht die Mühe zu verstehen), was er da tut! Fehler sind vorprogrammiert!

- Redundante Kommentare:

```
public int i; // neues i
for(i=0;i<10;i++)
// fuer alle i
```

Was würden Sie Ihrem Kollegen erklären (müssen), wenn Sie ihm/ihr den Code vorstellen?

Wiederholen Sie nicht, was der Code tut (das kann ich ja selbst lesen), sondern **beschreiben Sie, was der Code tun *sollte* und *warum***.

Beschreiben Sie dabei auch das Konzept hinter einem Codebaustein und gerne auch die typische Anwendung der Methode oder Klasse.

6 Code Smells: Schlechte Kommentare II

- Veraltete Kommentare

  Hinweis auf unsauberes Arbeiten: Oft wird im Zuge der Überarbeitung von Code-Stellen vergessen, auch den Kommentar anzupassen! Sollte beim Lesen extrem misstrauisch machen.

- Auskommentierter Code

  Da ist jemand seiner Sache unsicher bzw. hat eine Überarbeitung nicht abgeschlossen. Die Chance, dass sich der restliche Code im Laufe der Zeit so verändert, dass der auskommentierte Code nicht mehr (richtig) läuft, ist groß! Auskommentierter Code ist gefährlich und dank Versionskontrolle absolut überflüssig!

- Kommentare erscheinen zwingend nötig

  Häufig ein Hinweis auf ungeeignete Wahl der Namen (Klassen, Methoden, Attribute) und/oder auf ein ungeeignetes Abstraktionsniveau (beispielsweise Nichtbeachtung des Prinzips der "*Single Responsibility*")!

  Der Code soll im **Normalfall** für sich selbst sprechen: **WAS** wird gemacht. Der Kommentar erklärt im Normalfall, **WARUM** der Code das machen soll und die Konzepte und Design-Entscheidungen dahinter.

- Unangemessene Information, z.B. Änderungshistorien

  Hinweise wie "wer hat wann was geändert" gehören in das Versionskontroll- oder ins Issue-Tracking-System. Die Änderung ist im Code sowieso nicht mehr sichtbar/nachvollziehbar!

7 Code Smells: Schlechte Namen und fehlende Kapselung

```
public class Studi extends Person {
    public String n;
    public int c;

    public void prtIf() { ... }
}
```

Nach drei Wochen fragen Sie sich, was `n` oder `c` oder `Studi#prtIf()` wohl sein könnte! (Ein anderer Programmierer fragt sich das schon beim **ersten** Lesen.) Klassen und Methoden sollten sich erwartungsgemäß verhalten.

Wenn Dinge öffentlich angeboten werden, muss man damit rechnen, dass andere darauf zugreifen. D.h. man kann nicht mehr so einfach Dinge wie die interne Repräsentation oder die Art der Berechnung austauschen! Öffentliche Dinge gehören zur Schnittstelle und damit Teil des "Vertrags" mit den Nutzern!

- Programmierprinzip "**Prinzip der minimalen Verwunderung**"
  - Klassen und Methoden sollten sich erwartungsgemäß verhalten
  - Gute Namen ersparen das Lesen der Dokumentation
- Programmierprinzip "**Kapselung/Information Hiding**"
  - Möglichst schlanke öffentliche Schnittstelle
  - => "Vertrag" mit Nutzern der Klasse!

8 Code Smells: Duplizierter Code

```
public class Studi {
    public String getName() { return name; }
    public String getAddress() {
        return strasse+", "+plz+" "+stadt;
    }

```

```
7       public String getStudiAsString() {
8           return name+" ("+strasse+", "+plz+" "+stadt+")";
9       }
10  }
```

- Programmierprinzip "**DRY**" => "Don't repeat yourself!"

Im Beispiel wird das Formatieren der Adresse mehrfach identisch implementiert, d.h. duplizierter Code. Auslagern in eigene Methode und aufrufen!

Kopierter/duplizierter Code ist problematisch:

- Spätere Änderungen müssen an mehreren Stellen vorgenommen werden
- Lesbarkeit/Orientierung im Code wird erschwert (Analogie: Reihenhaussiedlung)
- Verpasste Gelegenheit für sinnvolle Abstraktion!

9 Code Smells: Langer Code

- Lange Klassen
  - Faustregel: 5 Bildschirmseiten sind viel
- Lange Methoden
  - Faustregel: 1 Bildschirmseite
  - (Martin 2009): deutlich weniger als 20 Zeilen
- Lange Parameterlisten
  - Faustregel: max. 3 … 5 Parameter
  - (Martin 2009): 0 Parameter ideal, ab 3 Parameter gute Begründung nötig
- Tief verschachtelte `if`/`then`-Bedingungen
  - Faustregel: 2 … 3 Einrückungsebenen sind viel

- Programmierprinzip "**Single Responsibility**"

  Jede Klasse ist für genau **einen Aspekt** des Gesamtsystems verantwortlich

9.1 Lesbarkeit und Übersichtlichkeit leiden

- Der Mensch kann sich nur begrenzt viele Dinge im Kurzzeitgedächtnis merken
- Klassen, die länger als 5 Bildschirmseiten sind, erfordern viel Hin- und Her-Scrollen, dito für lange Methoden
- Lange Methoden sind schwer verständlich (erledigen viele Dinge?)
- Mehr als 3 Parameter kann sich kaum jemand merken, vor allem beim Aufruf von Methoden
- Die Testbarkeit wird bei zu komplexen Methoden/Klassen und vielen Parametern sehr erschwert
- Große Dateien verleiten (auch mangels Übersichtlichkeit) dazu, neuen Code ebenfalls schluderig zu gliedern

9.2 Langer Code deutet auch auf eine Verletzung des Prinzips der Single Responsibility hin

- Klassen fassen evtl. nicht zusammengehörende Dinge zusammen

```
public class Student {
    private String name;
    private String phoneAreaCode;
    private String phoneNumber;

    public void printStudentInfo() {
        System.out.println("name:    " + name);
        System.out.println("contact: " + phoneAreaCode + "/" + phoneNumber);
    }
}
```

Warum sollte sich die Klasse `Student` um die Einzelheiten des Aufbaus einer Telefonnummer kümmern? Das Prinzip der "*Single Responsibility*" wird hier verletzt!

- Methoden erledigen vermutlich mehr als nur eine Aufgabe

```
public void credits() {
    for (Student s : students) {
        if (s.hasSemesterFinished()) {
            ECTS c = calculateEcts(s);
            s.setEctsSum(c);
        }
    }
}

// Diese Methode erledigt 4 Dinge: Iteration, Abfrage, Berechnung, Setzen ...
```

=> Erklären Sie die Methode jemandem. Wenn dabei das Wort "und" vorkommt, macht die Methode höchstwahrscheinlich zu viel!

- Viele Parameter bedeuten oft fehlende Datenabstraktion

```
Circle makeCircle(int x, int y, int radius);
Circle makeCircle(Point center, int radius);  // besser!
```

10 Code Smells: Feature Neid

```
public class CreditsCalculator {
    public ECTS calculateEcts(Student s) {
        int semester = s.getSemester();
        int workload = s.getCurrentWorkload();
        int nrModuls = s.getNumberOfModuls();
        int total = Math.min(30, workload);
        int extra = Math.max(0, total - 30);
        if (semester < 5) {
```

```
9                extra = extra * nrModuls;
10           }
11           return new ECTS(total + extra);
12       }
13   }
```

- Zugriff auf (viele) Interna der anderen Klasse! => Hohe Kopplung der Klassen!
- Methode `CreditsCalculator#calculateEcts()` “möchte” eigentlich in `Student` sein …

11 Weiterführende Links

- “Foundations: Clean Code” (The Odin Project)
- “Documentation Best Practices” (Google Styleguide)

12 Wrap-Up

- Code entsteht nicht zum Selbstzweck => Lesbarkeit ist wichtig

- Code Smells: Code führt zu möglichen (späteren) Problemen
  - Richtiges Kommentieren und Dokumentieren

    In dieser Sitzung haben wir vor allem auf Kommentare geschaut. Zum Thema Dokumentieren siehe die Einheit zu “Javadoc”.
  - Einhalten von Coding Conventions
    * Regeln zu Schreibweisen und Layout
    * Leerzeichen, Einrückung, Klammern
    * Zeilenlänge, Umbrüche
    * Kommentare
  - Einhalten von Prinzipien des objektorientierten Programmierens
    * Jede Klasse ist für genau **einen** Aspekt des Systems verantwortlich. (*Single Responsibility*)
    * Keine Code-Duplizierung! (*DRY* - Don’t repeat yourself)
    * Klassen und Methoden sollten sich erwartungsgemäß verhalten
    * Kapselung: Möglichst wenig öffentlich zugänglich machen

13 Zum Nachlesen

- Martin (2009)
- Passig und Jander (2013)
- Inden (2013, Kap. 10)

Lernziele

- k2: Ich kann typische Programmierprinzipien wie ‘Information Hiding’, ‘DRY’, ‘Single Responsibility’ erklären
- k3: Ich kann typische Code Smells erkennen und vermeiden
- k3: Ich kann leicht lesbaren von schwer lesbarem Code unterscheiden
- k3: Ich kann Programmierprinzipien anwenden, um den Code sauberer zu gestalten
- k3: Ich kann sinnvolle Kommentare schreiben

Quellen

Fowler, M. 2011. *Refactoring*. Addison-Wesley.
Inden, M. 2013. *Der Weg zum Java-Profi*. 2. Aufl. dpunkt.verlag.
Martin, R. 2009. *Clean Code*. mitp.
Passig, K., und J. Jander. 2013. *Weniger schlecht programmieren*. O’Reilly.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Exceptions:

- (Fowler 2011, p. 15)

> **Last modified:** 5eb5a0a (lecture: improve learning outcomes (Smells), 2025-05-21)